

Earning Autonomy: Learning and Applying an Organizer’s Decision Policies in Simulated Group Coordination

Joshua H. Kung Department of Computer Science, Columbia University jhk2214@columbia.edu

Advisor: Lydia B. Chilton, Computational Design Lab, Columbia University

Final Report — Distributed REsearch Apprenticeships for Master’s (DREAM), CRA-WP, Summer 2026

Abstract

Agentic systems can execute tasks competently but cannot yet execute them the way a particular person would. We present a system that learns an individual organizer’s coordination decisions as explicit, inspectable policies of the form `condition → action → rationale`, graduates each policy along an autonomy ladder from proposed to confirming to autonomous, demotes any policy the user overrides, and escalates situations that are novel or that contradict an existing policy rather than generalizing silently from a single event. The system is domain-agnostic: a rendering and decision engine that never encounters domain vocabulary, plus a thin adapter carrying everything task-specific. We evaluate it in a multi-agent simulation testbed built on scheduled agents operating over a shared state file, in which simulated participants hold private motives withheld from the organizer’s view, misreport their intentions, hold out for better offers, and renege late. We instantiate the system on running a 3v3 intramural basketball team across a full simulated season. We report four findings: that a decision interface earns its value only in the *hard region* of a decision, so information density should track local difficulty rather than spread uniformly; that when the system’s own ranking metric cannot separate the candidates at the boundary, stating the indifference is more useful than manufacturing a recommendation; that composition constraints make candidate value *marginal-given-need* rather than absolute; and that domain-generalizability is a testable property, which we test by relabeling the system to an unrelated coordination task through an adapter swap alone. The escalation-to-policy loop runs end-to-end and produced four tracked policies across the season. No user study has been conducted, and we are explicit about what that prevents us from claiming.

1. Introduction

AI agents have become competent at execution. Given a clear instruction, a modern agent will send the message, book the slot, or fill the form. What remains difficult is doing these things the way a particular person would have done them, and the gap is widest in the small recurring decisions that fill a week: who to put on a project, who to ask first, whether to push a meeting or drop the person who keeps missing it. These are not tasks where the instruction is the substance. The judgment is the substance, and delegating them requires the agent to know not only what the user would do but why.

This project takes up a narrower question than whether agents can act autonomously: **can an agent learn a specific person’s decision-making well enough to earn the right to act for them — incrementally, auditably, and without quietly going wrong?**

Three commitments follow from taking that question seriously.

First, learned rules must carry their *rationale*, not just their action. The same observed choice admits many explanations — the organizer picked the strongest candidate because they value raw quality, or because they needed a specific capability, or because that person apologized last week — and a rule learned with the wrong reason will be confidently wrong the first time the situations diverge.

Second, autonomy must be earned per-decision rather than granted globally. A rule that appears once has not demonstrated anything. It should climb a ladder as it is confirmed and fall back the moment it is overridden.

Third, and least obviously: the mechanism that decides when to act independently and the mechanism that decides when to interrupt the human are the same mechanism. We initially treated these as opposed — one implies asking less, the other asking more — and they are not. A detector for situations that are novel or that contradict existing rules is exactly the gate governing when autonomy should escalate. An agent without it is not more autonomous; it is less careful.

Building the interface for such a system surfaced a fourth question we had not anticipated, and which occupied the second half of the project. *Given* a ranking the agent has produced, where in the decision should the interface actually do work? The default answer, implicit in most decision-support UIs, is everywhere: render all candidates with all

attributes and let the user sort it out. We argue this is wrong in a specific and correctable way, and Section 6 develops the alternative.

Contributions.

1. A domain-agnostic engine for policy induction and graduated autonomy, with a verified adapter seam (Sections 4, 6.4).
2. A multi-agent simulation testbed in which participants have private motives, misreport, and renege — the conditions under which naive automation fails (Section 5).
3. The hard-region argument: a decision interface should allocate information density in proportion to local decision difficulty, and should decline to recommend when its own metric is indifferent (Sections 6.1, 6.2).
4. Constraints-as-shape: when a selection must satisfy composition requirements, candidate value becomes marginal-given-need, and constraint relaxation should be escalated rather than performed silently (Section 6.3).

We are explicit throughout about the boundary between what runs, what is implemented but not integrated, and what remains unbuilt. Section 9 states this plainly.

2. Related Work

Mixed-initiative interaction. The question of how automation and direct manipulation should share control is long-standing. Horvitz’s principles of mixed-initiative user interfaces [1] argue that an intelligent system should reason about the expected value of acting versus asking, and should time its interventions rather than assume more automation is better. The Lookout scheduling system made this concrete: infer the user’s intent, and open a dialog when inference is uncertain. Our escalation gate inherits this framing directly. Our cut-line design narrows it further: rather than asking when the system should intervene in an ongoing task, we ask *where within a single decision* the interface should spend its budget, and argue that the answer is the contested boundary rather than the decision as a whole. Horvitz’s related notion of the expected value of displayed information is the closest antecedent to what we call difficulty-driven information density.

Delegation and trust calibration. Whether people accept an agent that acts for them depends less on capability than on calibrated reliance. Lee and See [2] distinguish misuse from disuse and argue that appropriate reliance requires a system to make its purpose, process, and performance legible. Their model motivates two design commitments here. The autonomy ladder exists so that reliance is built incrementally against observed behavior rather than asserted up front. And the refusal to fake a recommendation (Section 6.2) follows from the same logic: a system that always renders confident output degrades calibration, because the user cannot distinguish a strong recommendation from an arbitrary tie-break.

Agentic coordination with human oversight. DoubleAgents [3], developed in the same lab, is the immediate predecessor to this work and shares its setting. It embeds transparency and control in an agentic planning tool through user intervention, value-reflecting policies, state visualization, and uncertainty flagging for human coordination tasks, using a built-in respondent simulation so users can rehearse and refine policies before live use, and reports that participants who initially hesitated to delegate grew more reliant as they experienced those affordances. Our system shares the policy-and-flagging architecture and the simulation-first methodology. It differs in two ways: policies here are *induced from observed choices* and carry an explicit autonomy status that changes over time, rather than being authored by the user; and we examine a single constrained-selection decision in depth rather than an end-to-end workflow, which is what surfaces the interface findings in Section 6.

LLM simulation as a design testbed. Park et al. [4] demonstrated that LLM-driven agents in a persistent environment produce believable and partly emergent social behavior, and earlier work on populated prototypes [5] established simulation as a way to surface edge cases before deployment. We adopt this method for a narrower purpose. Our simulated participants exist not to be believable but to make the decision *hard* in the specific ways real coordination is hard: private motives, unreliable commitment, and renegeing. Section 8 argues that decision fidelity, not behavioral realism, is the appropriate target for a testbed of this kind.

3. The Coordination Setting

We needed a task where judgment is genuinely required and where naive automation would visibly fail. Everyday group coordination — assembling a team, staffing a shift, filling a table — has four properties that make it well suited, and the properties generalize past any particular instance.

Preferences are private. Participants do not state why they are actually in or out. The organizer infers it from

behavior over time, and the behavior can be strategic.

Commitment is unreliable. A “yes” is not a promise. People over-commit, stall, go quiet, and renege close to the event. A system that treats an RSVP as ground truth is modeling a different problem.

There is no infeasible state. The event happens regardless; the organizer never fails outright, only does better or worse. This makes coordination optimization-flavored rather than constraint-satisfaction-flavored, which means the system must *judge* rather than *solve*.

Participation is endogenous. Allocation decisions change who is willing to participate next time. Choosing the strongest available option today makes the passed-over candidates less likely to be available tomorrow. The decision and the decision environment are coupled.

Any coordinator stands in this structure — a manager staffing shifts, an instructor assigning office hours, a friend organizing a game. We instantiated the system across several such scenarios during development (a lab game night, a watch party with cost and travel constraints, a camping trip, office-hours staffing) before settling on running a 3v3 intramural basketball team as the primary instance, because its composition constraints are crisp and its reliability signal is legible across repeated events.

The fourth property is the one this work engages least, and we return to it in Sections 9 and 10.

4. System

4.1 Overview

The system separates a domain-agnostic engine from a thin per-task adapter. The engine handles policy representation, evaluation, escalation, and rendering, and never encounters domain vocabulary. The adapter holds everything task-specific: decision points, feature extraction, choice classification, and display labels. Core rendering runs on the Python standard library with no external dependencies.

4.2 Policy representation and the autonomy ladder

Learned rules take the form:

```
decision_point + condition → action + rationale
```

The rationale is stored as a first-class field rather than as a comment. This is the central design decision of the learning component. When the system proposes a rule back to the user for confirmation, the user is confirming the *reason*, not the move — which is what makes the rule generalize correctly to situations the original observation did not cover.

Each policy carries an autonomy status and counters:

Status	Meaning
flagged	Surfaced for review; not yet a rule
proposed	Induced from observation, awaiting first confirmation
confirming	Confirmed at least once; still asks before acting
auto	Acts independently within its condition
paused	Suspended after override or audit failure

Promotion follows repeated confirmation; a user override demotes the policy immediately. Autonomy is therefore held per-decision-point rather than granted to the agent as a whole, and it is always reversible by the ordinary act of disagreeing with the system.

4.3 Escalation

Before a policy is applied, a gate asks whether the current situation is *new* — an unfamiliar decision point, or one that contradicts a rule already held. If so, the system stops, flags, and asks, rather than appending a policy from a single event. The user’s resulting decision becomes a tracked policy entering the ladder at `proposed`.

This is the behavior the project is organized around, and it unifies the two goals that initially seemed opposed. Escalation is not an exception to autonomy; it is the mechanism that governs it. The system acts on the routine and

raises a hand on the unfamiliar.

A companion reflection pass audits new and changed policies, returns a proceed-or-hold verdict before action, and records intended-versus-applied behavior afterward, so that divergence between what a policy specified and what was executed is visible rather than silent.

4.4 Persistence

Three stores hold durable knowledge, scoped so that domain-specific facts stay local while general rules can be promoted:

- **Policies** — induced rules with rationale, ladder status, and confirmation/override counts.
- **Learnings** — durable facts and directives (for example, who owns equipment), with confidence and provenance.
- **Metrics** — proposed measurements that would verify whether a policy is working, together with recurring themes and dependencies mined from history.

Current-world state lives in a single JSON object: participants and their attributes, the candidate pool, a four-state RSVP field (`in / maybe / out / silent`), observed reliability as show and flake counts, composition rules, event history, and the message log. The four-state RSVP model matters: a non-reply is not a decline, and the system never converts one into the other.

Two fields exist specifically to make the simulation non-trivial. Hidden motives carry ground-truth attendance and bench-aversion traits that are withheld from the organizer's view, so the organizer's information is genuinely incomplete. Scripted conditions inject deterministic scenario events such as double bookings.

5. Simulation Testbed

5.1 Architecture

The testbed runs as a set of scheduled agents operating over a shared JSON state file. No real messaging occurs; the state file is the entire world.

The coordinator agent runs on a fixed interval. Each cycle it reads messages arriving since its cursor, evaluates applicable policies, and may draft **at most one** outbound message per run. Critically, a draft is written to a `pending_draft` field rather than appended to the message log, and every draft carries the policies that triggered it with their identifiers, labels, and reasons. Confirmation is chat-based: the draft is printed for the organizer to approve, skip, or edit. While a draft is pending, the coordinator will not produce another.

This structure is the boundary between proposing and acting, made mechanical. The agent cannot take a social action without passing through a point where the human can see the action and the reasoning that produced it.

Persona agents, one per simulated participant, run on a shorter interval. Each reads the state file, finds unread messages addressed to it, rolls a reply probability, responds in character, appends to the log, and advances its cursor. Personas are parameterized on response rate, opinion strength, and agency, which turned out to be the three dimensions that most affect how much coordination work the organizer has to do.

Because messages carry sequence numbers and each agent maintains its own cursor, the simulation produces genuinely staggered, asynchronous conversation rather than lockstep turn-taking. Silence accumulates the way it does in real coordination, which is what creates the pressure that makes a coordinator act at all.

5.2 Resolution

Three modules resolve what actually happens. Attendance resolution determines who appears, using stated RSVP, hidden reliability, scheduling conflicts, prior rotation promises, and benching history. A disruption module applies a late event — injury, illness, work, lateness, or an unexpected fill-in. Outcome resolution rolls a result from realized strength. All three mutate shared state under a file lock.

The design commitment here is that failure is part of the environment rather than an error condition. A rolled ankle in warmups and a “yes” that evaporates the night before are things the system must handle, not exceptions to be excluded from the model.

6. The Decision Interface

6.1 The hard region

The first version of the board answered a feasibility question: do we have enough people, equipment, a time? This is a binary check, and it is the easy half of the decision. The hard half is optimization — is this the best feasible set, and what is being traded off at the margin?

The initial demonstration scenario obscured this. It showed four familiar candidates for three slots: a decision with almost no entropy, where one option dominates and the organizer already knows everyone. We had built a decision-support tool and demonstrated it on precisely the class of problem where such a tool is unnecessary.

Designing the demonstration turned out to be part of designing the system. An instance is hard only when three conditions hold at once: enough candidates that the organizer cannot hold them in working memory; unfamiliarity, so learned signals are the only available tie-breakers; and a genuine fight at the boundary, where the numeric score and the reliability read disagree. We replaced the scenario with twelve candidates for six slots, an organizer who does not know them, and a near-tie at the cut.

The principle that followed: **difficulty drives information density**. The interface stays calm where the answer is obvious and expands — surfacing tie-break dimensions for the two or three contested candidates only — where it is not. The obvious selections need no interface. The contested slot needs all of it.

6.2 Refusing to fake a recommendation

Candidates are ordered not by raw score but by *value*, combining score with a reliability weight derived from observed show and flake history, so a slightly weaker but dependable candidate can outrank a stronger unreliable one.

The interesting behavior is at the cut. When the two candidates straddling the boundary fall within a small margin of each other in value, the system's own metric cannot separate them — a one-point gap on a hundred-point scale is inside the noise. The design response is to stop pretending. The interface surfaces a focused score-versus-reliability comparison for exactly those two, highlights which dimension favors whom, and, when the dimensions disagree and the value is level, states the indifference directly rather than emitting a recommendation.

This is not a failure state. It is the case where the user's judgment is the actual content of the decision, and a tool that renders confident output here misrepresents its own epistemic position — precisely the failure that degrades calibrated reliance [2].

The design further treats the user's unforced choice at such a tie as a *rule* rather than a click. The organizer is not merely selecting a name; they are expressing how they weigh dependability against raw quality at the margin, which is a reusable policy entering the ladder at `proposed`.

6.3 Constraints have a shape

A significant revision came from recognizing that “best N” is not a meaningful target once a selection has composition requirements. A team is not the top five by value; it is two guards, two forwards, and a center. The cut therefore stopped being top-N and became a per-position assignment.

The consequence: **value becomes marginal-given-need**. A higher-rated candidate for an already-filled position loses their slot to a lower-rated candidate who fills an empty one. “Best” is defined only relative to the shape being filled. The most instructive case in our scenario is a single-slot position where the numeric score favors one candidate with a poor reliability history and the behavioral read favors another who is dependable; because it is the only such slot, that one call determines whether the composition constraint is satisfiable at all.

Constraints are modeled in two tiers. Positions are hard by default. Only when a required position genuinely cannot be filled does the composition become infeasible and escalate to soft — a prompt to relax the requirement — following the same needs-your-call pattern as the cut-line tie. The system does not relax a constraint silently; it flags that the constraint is about to break and asks.

A related bug proved conceptual rather than cosmetic. Mid-sequence, with replies still arriving, the board fired its relax-the-constraint escalation because a position was unfilled — conflating *waiting* with *failing*. These are different states. If a live candidate for the open position remains un-asked, the correct action is to chase; only when the pool is exhausted should relaxation be proposed. Encoding the distinction matters because it corresponds to a real coordinator's judgment: one does not concede the shape of the team merely because the room is quiet.

Layout followed one principle: say each thing exactly once, and let hierarchy carry emphasis. The board separates a decision column — verdict, constraints, assignment, the single contested call — from a candidate column where per-person detail appears once. Satisfied constraints recede to a quiet line; the open or contested slot expands.

6.4 Generality as a testable property

The claim that an engine is domain-agnostic is easy to assert in a README and hard to verify. We treated it as falsifiable: can the system be relabeled to an unrelated coordination problem *without modifying engine code*, changing only the adapter and the state file?

The first attempt failed informatively. The renderer contained a sport-specific icon, the strings “the gym” and “on court,” and a “Best N” label compiled directly in. Reliability weights sat as constants in the display layer. The required participant count was hardcoded. Each was a place where the engine secretly knew the domain, and each had to move into the adapter or be read from the rules object.

After that pass, the identical rendering code produced a correct board for an office-hours staffing relabel, and the single-decision renderer drives a dinner-coordination adapter without modification. This is the strongest verified claim in this report, and it was made verifiable by a test that initially failed.

A secondary discipline supported it: every line on the board must be either **state** or **action**, with no standalone statistics. A line such as “Replies: 2 in, 2 out” reports without implying what to do. Applying the rule removed a surprising amount of content, and nearly everything it removed was domain-specific residue that had not been justified.

7. Results

We report what the system demonstrated. These are observations from a demonstration deployment, not a controlled evaluation; Section 9 states what that limits.

The escalation-to-policy loop runs end-to-end. Across a full simulated season, the system encountered situations with no governing rule, escalated them to the organizer rather than generalizing, and converted the organizer’s resulting decisions into tracked policies that entered the ladder. This is the core behavior the architecture exists to produce, and it is the finding the project most directly establishes.

Policies accumulated with differentiated autonomy. The season produced four tracked policies, of which two reached autonomous status and two remained in confirming — that is, the ladder discriminated rather than promoting everything uniformly. Two durable learnings and eight candidate metrics were also accumulated; the metrics remain proposals and none were promoted into tracked decision features.

Policy evaluation and the reflection gate execute as designed. The path from state through adapter feature extraction to policy evaluation to the pre-action gate resolves correctly, returning both a policy recommendation and a proceed-or-hold verdict.

The adapter seam is real. The relabel test described in Section 6.4 passes: the same rendering code serves basketball, office-hours staffing, and dinner coordination with only adapter and state changes.

Simulated participants produced unscripted strategic behavior. Agents who were passed over began holding out, negotiating, and going quiet. In one episode a participant demanded a guaranteed role as a condition of participating, received the guarantee, and then did not appear. None of this was authored. We report it as an illustrative observation rather than a result — it is a single episode with no controlled comparison — but it points directly at the endogenous-participation dynamic of Section 3, which the system does not yet model. That the dynamic emerged from incentives rather than from scripting is the strongest evidence we have that the testbed exercises the right pressures.

8. Discussion

Cloning the action is easy; cloning the reason is the whole problem. An early version happily learned “the organizer picks the strongest option” from a single instance. The same pick could have meant several incompatible things, and a rule carrying the wrong reason is confidently wrong the first time the situations diverge. Everything downstream — when to act, when to ask — depends on getting the *because* right. This is why rationale is a stored field and why confirmation is framed as confirming the reason.

A faithful clone reproduces the user’s mistakes. A system that learns a user’s rule will also apply it when the rule is wrong. This forces an explicit stance: is the system a mirror or a coach? We chose mirror-with-warning — act as the user would, but flag when their own rule appears likely to backfire. This is a design commitment rather than an implementation detail, and it is the substance of what “aligned to a user” means in this setting.

Decision fidelity beats behavioral realism. Considerable effort went into making simulated participants sound like

people before we recognized this was the wrong target. What matters is whether deciding inside the simulation exercises the same judgment the real task would. A simulation unrealistic in ways that do not affect the decision is fine; one that is lifelike but decision-trivial is not. This reframing motivated the demonstration redesign in Section 6.1: the problem with the original four-candidate scenario was not that it was unrealistic but that it was decision-trivial.

9. Limitations

We state these specifically, because the boundary between what runs and what is designed is the most important thing a reader needs to know.

No user study. The central claims are architectural and design-level. Whether learned policies actually match a user’s judgment, whether the autonomy ladder earns trust at an appropriate pace, whether users notice and value deliberate non-recommendation, and whether people can detect that the agent has inferred the wrong reason are all untested. This is the most significant gap and the obvious next step.

Results are from a demonstration, not an evaluation. Section 7 reports what the system did across one simulated season. There is no baseline, no comparison condition, and no quantitative measure of decision quality.

Orchestration is not self-contained. The individual resolution stages are implemented and each mutates state correctly, but no single module sequences them. Locking the planned lineup, converting attendance into a realized lineup, and advancing the season are specified in an operating playbook rather than in code.

Persona agents are scheduled tasks, not repository artifacts. The simulated participants run as scheduled agents configured in the author’s account rather than as programs in the repository, so the simulation is not reproducible from the repository alone.

Two subsystems are not wired to the main path. An event-sourced state model correctly derives reliability, RSVPs, and standings from an append-only log, but the renderers still read the monolithic state file directly. Separately, a mechanism module models outreach as an intervention with an estimated show probability, bench sensitivity, perceived playing chances, and accrued future obligation, and a variants renderer compares three outreach strategies against it — but neither feeds the primary decision board.

The cut-line tie-break does not yet write back. Section 6.2 describes the user’s boundary choice as minting a reusable rule. The interface expresses this; the write into the policy store is not implemented.

Two policies are unreachable through the adapter. The adapter’s declared decision points omit two points that existing policies reference, so those policies cannot currently be reached through the normal flow.

De-hardcoding is incomplete. Reliability weights remain constants in several view and adapter paths despite the argument of Section 6.4, and resource detection still infers ownership by searching learning text rather than consulting the adapter.

No evaluation infrastructure. There is no automated test suite, no schema validation, and no continuous integration.

Cross-domain transfer is untested. Section 6.4 verifies that the *engine* is domain-independent. It does not show that a policy induced in one domain usefully transfers to another. That is the stronger claim, and it remains unmade.

Endogenous participation is essentially unmodeled. Of the four properties motivating the setting, three are expressed by the system. The fourth — that allocation decisions change future willingness to participate — is the one we identified as most interesting and engaged least.

10. Future Work

Evaluate with users. Measure whether learned policies match the judgment of the person they were induced from, whether the cut-line design changes decision quality or confidence relative to a conventional ranked list, and whether users can detect when the system has inferred the wrong reason.

Close the endogenous loop. Model invitation as a costly intervention: that inviting someone as a fallback raises their probability of declining later, that being benched accrues obligation, and that the framing and ordering of outreach change the pool available next time. This converts the setting from a sequence of independent selections into one where the decision alters the decision environment — the property that makes coordination genuinely hard, and the one the unscripted holdout behavior in Section 7 points at.

Test transfer, not just relabeling. Demonstrate that a policy induced in one coordination task measurably helps in

an unrelated one.

Distinguish novelty from stakes. Present escalation is structural — a situation is new or it contradicts a rule. A more useful system would model consequence, so it interrupts for decisions that matter rather than merely for decisions that are unfamiliar.

Study failure at the edges. The most informative cases are those where the user’s own policy is internally inconsistent, or where two learned rules collide. These are where delegation actually breaks, and where an agent’s willingness to say “I do not know what you would want here” is worth more than its ability to act.

11. Conclusion

We set out to test whether an agent can learn a specific person’s decision-making well enough to earn the right to act for them. The architecture we arrived at holds autonomy per-decision rather than globally, stores the reason alongside the action because the reason is what generalizes, and treats escalation on unfamiliar situations as the same mechanism that governs autonomy rather than as its opposite. Across a full simulated season, that loop ran end-to-end: novel situations escalated, the organizer’s calls became tracked policies, and those policies graduated at different rates.

Building the interface for that system produced a second argument we did not expect. A decision-support interface should be organized around the difficulty of the decision rather than the completeness of the data. Most of a selection is settled, and the interface should get out of its way; the decision lives at the cut line, and that is where information density belongs. When the system’s own metric cannot separate the candidates at that boundary, the honest and more useful behavior is to say so, and to treat the user’s resulting call as a rule worth remembering.

We have not evaluated any of this with users, and we have not modeled the property of coordination we consider most interesting — that today’s decision changes the environment in which tomorrow’s is made. The simulated participants demonstrated that property unprompted, which is both the most encouraging result and the clearest statement of what to build next.

The long-term goal is not an agent that does more. It is an agent that has earned enough of a person’s judgment to operate with less supervision, and that knows reliably when it has not.

Acknowledgments

This work was supported by the CRA-WP Distributed REsearch Apprenticeships for Master’s (DREAM) program. I thank Professor Lydia B. Chilton for advising this project throughout the summer, and the members of the Computational Design Lab at Columbia for feedback on early versions of the system and interface.

References

- [1] E. Horvitz. Principles of Mixed-Initiative User Interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI ’99)*, 159–166, 1999. doi:10.1145/302979.303030
- [2] J. D. Lee and K. A. See. Trust in Automation: Designing for Appropriate Reliance. *Human Factors*, 46(1):50–80, 2004. doi:10.1518/hfes.46.1.50_30392
- [3] T. Long, X. Zhang, S. Wang, Z. Yu, and L. B. Chilton. DoubleAgents: Human-Agent Alignment in a Socially Embedded Workflow. arXiv:2509.12626.
- [4] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST ’23)*, 2023. doi:10.1145/3586183.3606763
- [5] J. S. Park, L. Popowski, C. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Social Simulacra: Creating Populated Prototypes for Social Computing Systems. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology (UIST ’22)*, 2022. doi:10.1145/3526113.3545616